



TrafficTelligence:

Advanced Traffic Volume Estimation with Machine Learning is a machine learning-based web application designed to predict traffic volume using environmental and temporal data. Here's a brief overview:



Purpose

To help reduce urban traffic congestion through predictive modeling that supports:

- Dynamic traffic signal control
 - Urban infrastructure planning
 - Smarter navigation for commuters
-



Dataset Features

- **Holiday** (binary)
 - **Temperature** (°C)
 - **Rain/Snow** (mm)
 - **Date & Time** (parsed into year, month, day, hour)
 - **Traffic_volume** (target variable)
-



ML Techniques

- **Preprocessing:** Handling missing data, scaling, encoding
- **Models Used:**
 - Random Forest Regressor (**Best performer**)
 - XGBoost Regressor
- **Evaluation Metrics:**
 - Random Forest: $R^2 = 0.8489$, RMSE = 772.73

- XGBoost: $R^2 = 0.8433$, RMSE = 787.06
-



Tech Stack

- **Frontend:** HTML, CSS
 - **Backend:** Python, Flask
 - **Libraries:** pandas, numpy, scikit-learn, xgboost
 - **Deployment:** Flask app running on localhost
-



How to Run

bash

Copy code

```
pip install -r requirements.txt
```

```
python model_training.py
```

```
python app.py
```



Main Features of TrafficTelligence



Machine Learning Features

- **Traffic Volume Prediction** using historical environmental and temporal data
- **Feature Engineering:**
 - Extracts **hour, day, month, year** from timestamps
 - Uses weather data: temperature, rain, snow
 - Identifies holidays as a binary feature
- **Model Selection:**
 - Trains and compares **Random Forest** and **XGBoost** regressors
 - Selects best-performing model (Random Forest) based on R^2 and RMSE



Data Processing Features

- **Missing Value Handling** using SimpleImputer
- **Feature Scaling** using StandardScaler
- **Categorical Encoding** where applicable
- **Data Visualization** using matplotlib and seaborn



Web Application Features

- **Flask-based Web Interface** for real-time traffic volume prediction
- **User Input Form** to enter environmental and date-time values
- **Real-time Prediction Output** based on trained model
- **Responsive Design** using HTML5/CSS3



Project & Deployment Features

- **Modular Codebase:**
 - app.py for web server
 - model_training.py for ML training
- **Model Persistence** via model.pkl and scaler.pkl
- **Local Deployment** using Flask server